

# Advanced JS

## Orientation Session : [0]

→ (Basics)<sup>only</sup> bas JS ال قسور

- What we miss :

\* - pros & cons of every solution

\* Before & After learning

DS Algorithms

How did you solve it?

How much it got improved?

- How to write prompt? Prompt Engineering, Temperature Tokens, ...

- Integrate with OpenAI, Gemini

- You already know the Base of Javascript and you ~~get~~ produce an output, but if you got more Deep, you would get a better app with higher performance and better security.

- if you created an object, and don't need it anymore, you'll leave it? That will fill the memory and as a result the performance will go down ↓

→ Garbage Collector, Hidden Class, String pool

→ Keys order inside the object, methods creation in objects?

→ Security? token in Local storage?

get token & verify it

↓  
null

↓  
undefined

↓  
" "

→ hole in arrays → how does it affect performance?  
-----

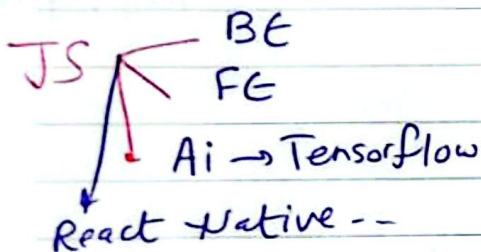
→ Somethings you use it in code, but you don't know that it can make a problem after that.

→ Design Patterns, Solid Principles - SOLID

→ Companies look for Devs/ Engineers have very good Knowledge of Basics + Prompt Engineering

↓  
How it work?

↓  
Best one?



Multiple Solutions

Out put → which o/p is better?  
Difference

Mindset Difference?

→ Why the Problem happened and solve it from its roots not ~~at~~ only solve it.

→ Tracing Code → ✓ Why that problem happened?  
not How it will be solved?

→ That's the difference between a

Coder | Developer (Engineer)

not always workarounds → why the char is lower case?



→ So, I have to know how to get/catch the problem from its root

and know when I can say that to solve that issue → That will cost me.

or only make a ~~hot~~ hot fix now, what It will do?

production issue ↗ hot fix?  
↘ from its root?

→ get Back to the Business! Based on my situation

→ Even if you are a Junior, You are assigned to do that!

↳ I've done 1, 2, 3 -- bec. it has a low cost

So everyone will get impressed! Hurray!!

2nd part

if I am ~~holding~~ holding my bag with all my luggage, and I am searching for a thing <sup>certain</sup>

Anagram // name .... // mean <sup>not best solution</sup>

function is Anagram (a, b) ?

// name == ['n', 'i', 'a', 'm', 'e'] ⇒ ['a', 'e', 'i', 'm', 'n'] ⇒ aemn

// mean == ['m', 'e', 'a', 'n', 'i'] ⇒ ['a', 'e', 'i', 'm', 'n'] ⇒ aemn

return a.split('').sort().join('') == aemn

== b.split('').sort().join('')

Sort  $n \log n$

# DSA ~ 1 month

fast comparison

## 1) Normal for loop

const arr = [1, 2, 3, 4, 5]

```
for (let i = 0; i < arr.length; i++) {  
  console.log(i, arr[i]);  
}
```

?

→ You control start, condition, inc/dec.

→ Best if you need index

|   |   |
|---|---|
| 0 | 1 |
| 1 | 2 |
| 2 | 3 |
| 3 | 4 |
| 4 | 5 |

## 2) for ... of

```
for (let item of arr) {  
  console.log(item)  
}
```

?

- This directly gives you the value.
- No indexes involved automatically
- Best if you only need values.

~



### Basic Approach (using object as frequency map)

11 Count chars from a  
for (const char of a) {  
    freq[char] = (freq[char] || 0) + 1;  
}

freq = {  
a: 1  
e: 1  
m: 1  
h: 1

7

 $O(n)$ 

## Built-in methods.

1)  $\rightarrow$  Constant space

here we are ~~not~~ looping on ~~all~~  
all elements, we ~~only~~ increment  
the desired freq. first

then check the index of each char direction

→ we can use an array of size 26 to count the freq of chars (assuming only lowercase letters)

$[0, 0, \dots] \Rightarrow \text{Size}$   
 1- Name  $[1, 0, 0, 0, 1, \dots]$ ,  $[1, 0, 0, 0, 1, \dots]$

$$9 = 0 + 0 ?$$

Another example

in the array?

target = 9, arr = [2, 3, 5, 7]

$O(n^2)$   $\Rightarrow$  Brute force approach X XX

Optimized approach  $O(n)$  ✓ ✓ ✓ ✓

```
function hasPairsWithSum(arr, target) {
  const map = {};
  for (let i = 0; i < arr.length; i++) {
    const diff = target - arr[i];
    if (map[diff] !== undefined) {
      return map[diff];
    }
    map[arr[i]] = i;
  }
}
```

Target: Why we write  $a_i$  for what?

$a_i$  is the value of  $a_i$



→ Error handling Skill B✓!

→ Is JS blocking or non blocking?

↳ Blocking by default → one call stack  
one main Thread

→ executes code line by line

~~But~~ But it also can do non-blocking things.

Using ⇒ setTimeout, fetch, promises, async/await  
event listeners.

Why didn't setTimeout block?

→ browsers / Web APIs handle it outside the main JS thread

→ Then callback<sup>fn</sup> goes to the queue

↳ console.log("B") -- for example

→ Event loop pushes it later to call stack.

So JavaScript runtime environments support non-blocking async operations.

timers, network, DOM events → help prepare things<sup>3</sup>  
in background all  
it notify JS later.

tick

⇒ one cycle of the Event loop